

08

Lecture 08

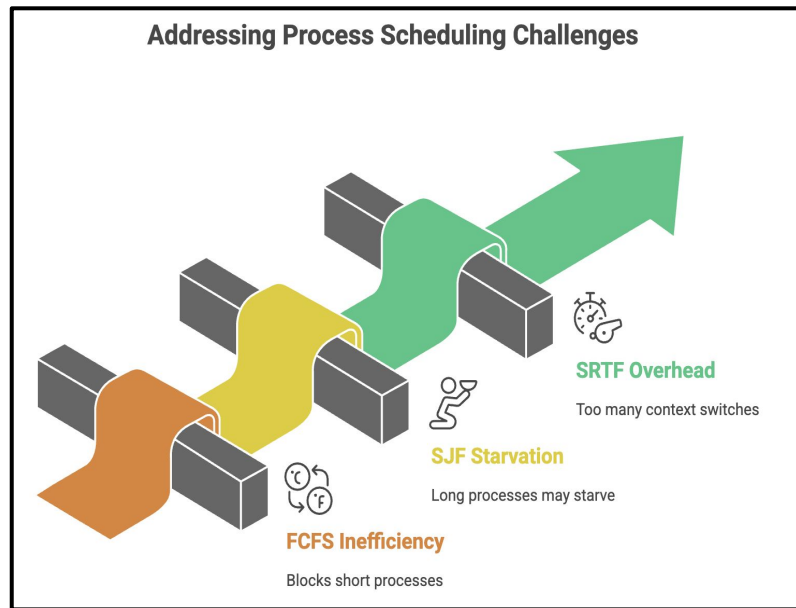
**CPU Scheduling Algorithms II: Round Robin
and Priority**

by - Brijesh Shukla
C07: Operating Systems

Join the lecture online on your dashboard.

Quick Review: FCFS, SJF, SRTF:

Algorithm	Preemption	Avg Wait	Problem
FCFS	No	Poor	Convoy Effect
SJF	No	Optimal	Starvation
SRTF	Yes	Best	High Overhead



✗ **All three:** Not truly fair to all processes

The Coffee Shop: New Challenges:

Same Setup:

Customer A (arrives at 9:00 AM) → Orders 10 lattes → Takes 10 minutes

Customer B (arrives at 9:01 AM) → Orders 1 espresso → Takes 1 minute

Customer C (arrives at 9:02 AM) → Orders 1 cappuccino → Takes 1 minute

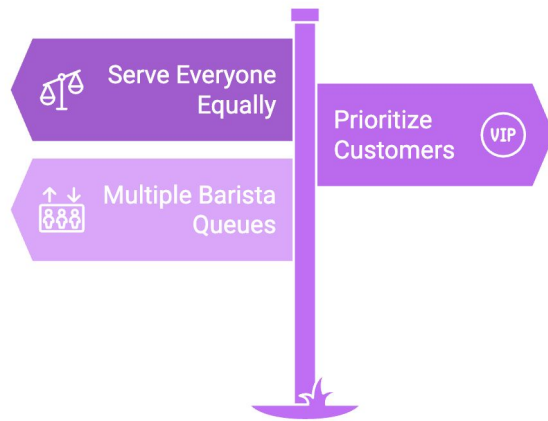
New Complications:

 Customer D (9:03 AM) → VIP Member → 2 minutes

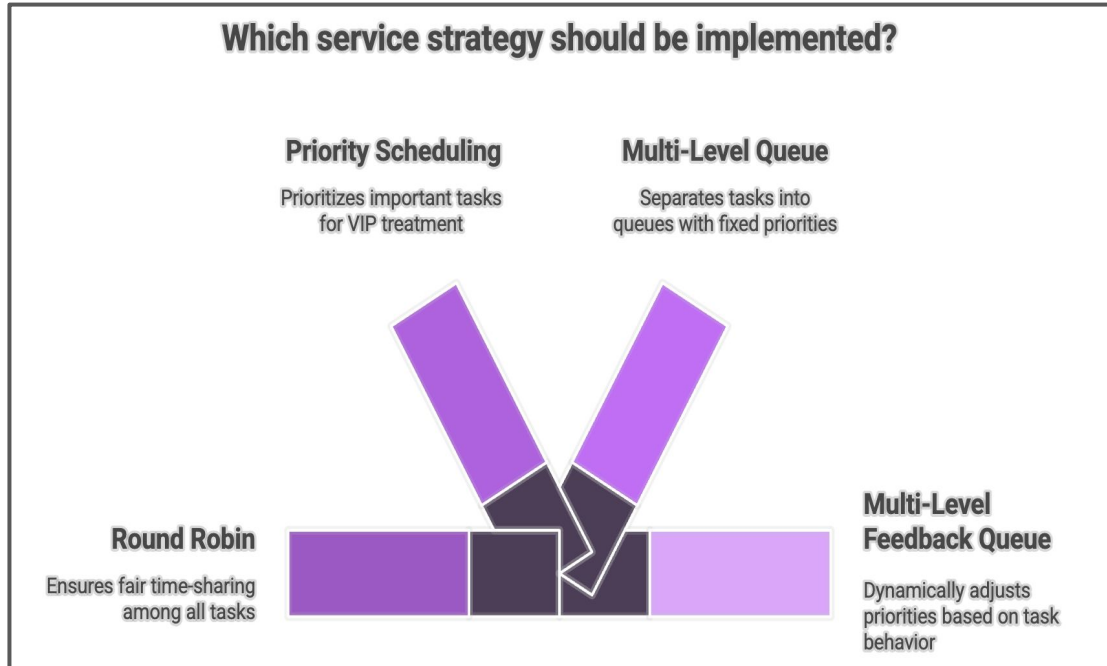
 Customer E (9:04 AM) → Regular → 5 minutes

 Customer F (9:05 AM) → Student discount → 3 minutes

How should we manage customer service?



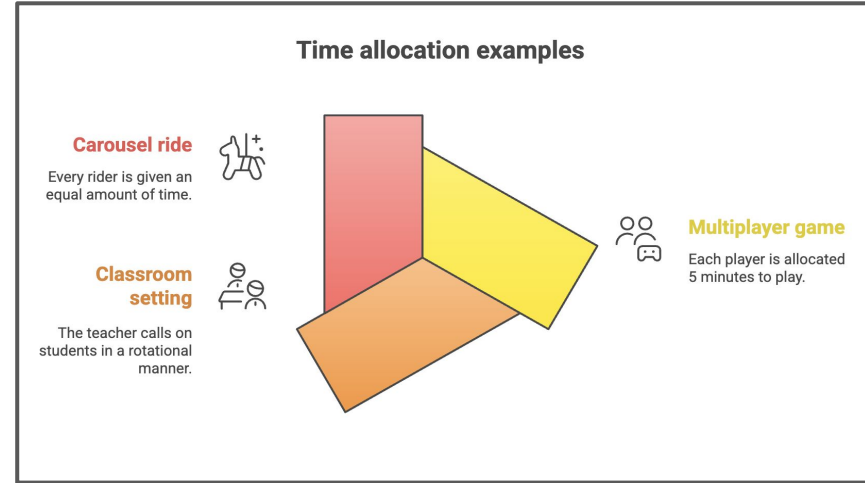
The Coffee Shop Analogy:



Round Robin (RR) Algorithm - "Everyone Gets a Turn" Algorithm:

Core Principle:

- Each process gets a small unit of CPU time (time quantum)
- After quantum expires, process is preempted
- Process goes to back of ready queue
- CPU scheduler picks next process



Mechanism:

- **Criteria:** Time Quantum (q).
- **Mode:** Preemptive (Timer interrupt forces switch).
- **Data Structure:** Circular FIFO Queue.

Round Robin (RR) Algorithm - "Everyone Gets a Turn" Algorithm:

Round Robin Algorithm Process:

Step 1: Set time quantum (q) - typically 10-100 ms

Step 2: Maintain Ready Queue (FIFO circular)

Step 3: Pick process at head of queue

Step 4: Execute for time quantum OR until completion

Step 5: If process not done:
→ Move to back of queue
→ Pick next process

Step 6: Repeat

Process Timing Acronyms	
Process Id	Pid
Arrival Time	AT
Burst Time	BT
Turnaround Time	TAT
Waiting Time	WT
Average Waiting Time	AWT

Example 1 - RR:

Pid	AT (ms)	BT (ms)
P0	0	10
P1	1	1
P2	2	1

Circular FIFO Queue

P0	P1	P2	P0	P0	P0	P0
----	----	----	----	----	----	----

Gantt Chart

P0	P1	P2	P0	P0	P0	P0	
0	2	3	4	6	8	10	12

Pid	AT	BT	CT	TAT = CT-AT	WT = TAT - BT
P0	0	10	12	$12 - 0 = 12$	$12 - 10 = 2$
P1	1	1	3	$3 - 1 = 2$	$02 - 1 = 1$
P2	2	1	4	$4 - 2 = 2$	$02 - 1 = 1$

$$AWT = (2 + 1 + 1) / 3 = 1.33 \text{ ms}$$

Example 2 - RR:

Pid	AT (ms)	BT (ms)
P0	0	5
P1	1	3
P2	2	1
P3	3	2
P4	4	3

Circular FIFO Queue

P0	P1	P2	P0	P3	P4	P1	P0	P4
----	----	----	----	----	----	----	----	----

Gantt Chart

P0	P1	P2	P0	P3	P4	P1	P0	P4	
0	2	4	5	7	9	11	12	13	14

Pid	AT	BT	CT	TAT = CT-AT	WT = TAT - BT
P0	0	5	13	$13 - 0 = 13$	$13 - 5 = 8$
P1	1	3	12	$12 - 1 = 11$	$11 - 3 = 8$
P2	2	1	5	$05 - 2 = 3$	$03 - 1 = 2$
P3	3	2	9	$09 - 3 = 6$	$06 - 2 = 4$
P4	4	3	14	$14 - 4 = 10$	$10 - 3 = 7$

$$AWT = (8 + 8 + 2 + 4 + 7) / 5$$

$$= 5.8 \text{ ms}$$

The Quantum Effect:

Quantum too Small ($q = 1\text{ms}$):

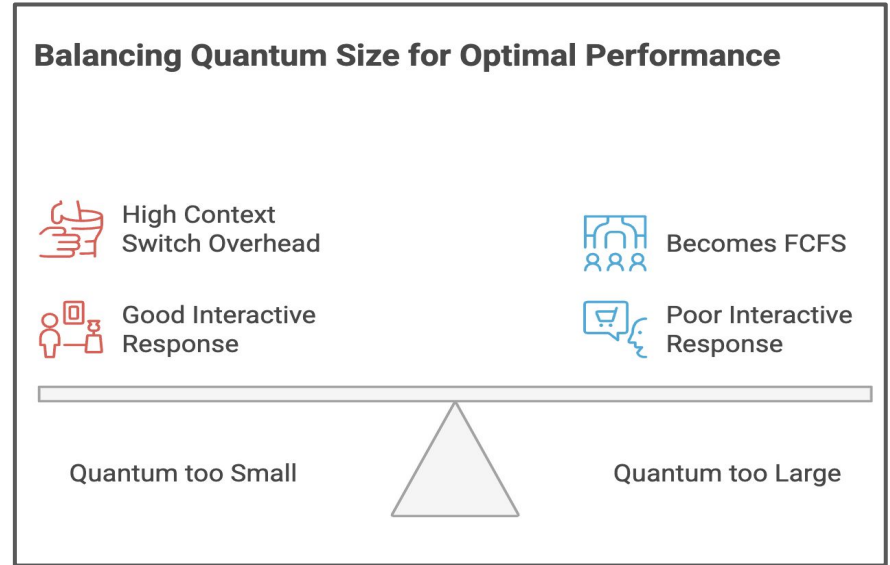
- Cafe Analogy: Barista spends more time switching cups than making coffee!
- Tech Term: High Context Switch Overhead.

Quantum too Large ($q = 10\text{ minutes}$ or $q = \infty$):

- Cafe Analogy: Customer A hogs the Barista. Customer B waits 10 mins.
- Tech Term: Becomes FCFS.

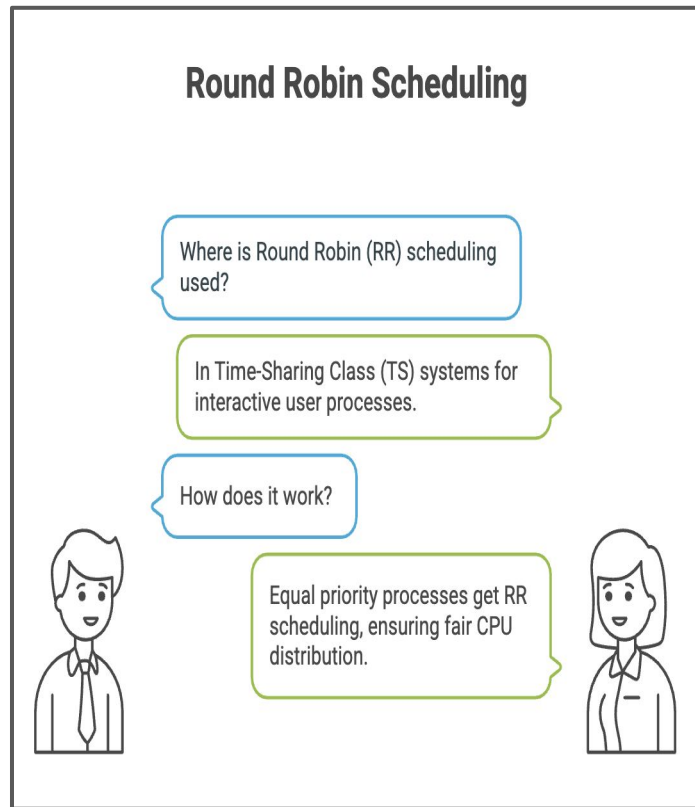
Perfect Quantum ($q = 10\text{-}100\text{ ms}$):

- Balance between fairness and efficiency
- Context switch overhead $< 1\%$ of quantum
- Good interactive response
- Most systems use: **10-20 ms**



Round Robin in macOS:

- **Interactive Apps (Standard Priority):**
 - When you run Safari, Spotify, and VS Code, macOS uses a Round Robin-style approach (within the same priority level).
 - It gives each app a tiny slice of time (Quantum) so they all feel "live."
- **The "Spinning Beach Ball":**
 - If an app exceeds its quantum repeatedly without yielding or responding, macOS might flag it as unresponsive.



When not to use Round Robin ?



- **✗ Not for batch processing (overhead)**
- **✗ Not for real-time systems (unpredictable delays)**

PRIORITY SCHEDULING - THE COFFEE SHOP VIP SYSTEM

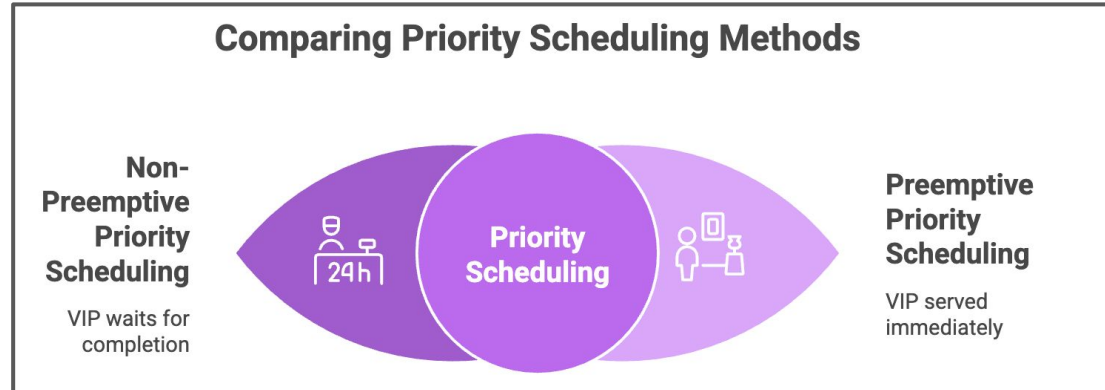
What if some customers are more important?

New Rule: VIP System

- 👑 VIP Customer (Priority 1) → Gets served first (CEO, urgent order)
- ☀️ Regular Customer (Priority 2) → Normal service
- 👤 Student Discount (Priority 3) → Lowest priority

Scenario:

- Regular customer ordering (Priority 2)
- VIP walks in (Priority 1)



PRIORITY SCHEDULING - THE "IMPORTANCE FIRST" ALGORITHM

Core Principle:

- Each process has a priority number
- CPU allocated to highest priority process
- **Lower number = Higher priority** (or vice versa)
- Can be Preemptive or Non-Preemptive

Mechanism:

- **Criteria:** Priority Value (assigned to each process)
- **Mode:** Can be Preemptive OR Non-Preemptive
- **Data Structure:** Priority Queue (Min-Heap or Max-Heap)

Non-Preemptive Priority Algorithm:

- Once CPU allocated, runs to completion
- Check priority only when process finishes
- Like: VIP waits for current customer to finish

Algorithm Process:

Step 1: Assign priority to each process (lower number = higher priority)

Step 2: Maintain Ready Queue sorted by priority

Step 3: When CPU is free, select process with HIGHEST priority

Step 4: Execute selected process to completion

Step 5: Remove from queue, reselect highest priority

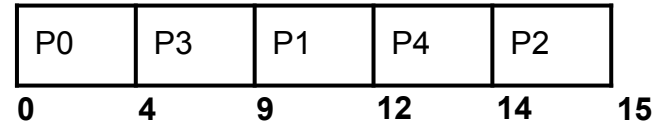
Step 6: Repeat

Process Timing Acronyms	
Process Id	Pid
Arrival Time	AT
Burst Time	BT
Turnaround Time	TAT
Waiting Time	WT
Average Waiting Time	AWT

Example: Non-Preemptive Priority Algorithm

Pid	AT (ms)	BT (ms)	Priority
P0	0	4	5
P1	1	3	2
P2	2	1	4
P3	3	5	1
P4	4	2	3

Gantt Chart



Pid	AT	BT	CT	TAT = CT-AT	WT = TAT - BT
P0	0	4	4	$04 - 0 = 4$	$04 - 4 = 0$
P1	1	3	12	$12 - 1 = 11$	$11 - 3 = 8$
P2	2	1	15	$15 - 2 = 13$	$13 - 1 = 12$
P3	3	5	9	$09 - 3 = 6$	$06 - 5 = 1$
P4	4	2	14	$14 - 4 = 10$	$10 - 2 = 8$

$$AWT = (0 + 8 + 12 + 1 + 8) / 5$$

$$= 5.8 \text{ ms}$$

Preemptive Priority Algorithm:

- Higher priority process can interrupt current process
- Check priority at every arrival
- Like: VIP cuts the line immediately

Algorithm Process:

Step 1: Assign priority to each process

Step 2: Maintain Ready Queue sorted by priority

Step 3: Select process with HIGHEST priority

Step 4: Execute process

Step 5: On new process arrival:
If new priority > current priority → Preempt
Else → Continue current process

Step 6: On process completion → Select next highest priority

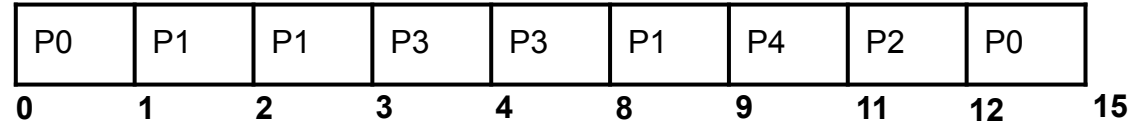
Step 7: Repeat

Process Timing Acronyms	
Process Id	Pid
Arrival Time	AT
Burst Time	BT
Turnaround Time	TAT
Waiting Time	WT
Average Waiting Time	AWT

Example: Preemptive Priority Algorithm

Pid	AT (ms)	BT (ms)	Priority
P0	0	4	5
P1	1	3	2
P2	2	1	4
P3	3	5	1
P4	4	2	3

Gantt Chart



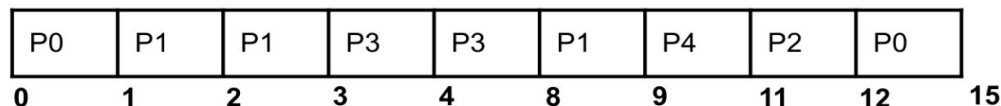
Pid	AT	BT	CT	TAT = CT-AT	WT = TAT - BT
P0	0	4	15	$15 - 0 = 15$	$15 - 4 = 11$
P1	1	3	09	$9 - 1 = 8$	$08 - 3 = 5$
P2	2	1	12	$12 - 2 = 10$	$10 - 1 = 9$
P3	3	5	08	$08 - 3 = 5$	$05 - 5 = 0$
P4	4	2	11	$11 - 4 = 7$	$07 - 2 = 5$

$$AWT = (11 + 5 + 9 + 0 + 5) / 5 = 6.0 \text{ ms}$$

Preemptive Priority Algorithm - STARVATION PROBLEM:

Pid	AT (ms)	BT (ms)	Priority
P0	0	4	5
P1	1	3	2
P2	2	1	4
P3	3	5	1
P4	4	2	3

Gantt Chart



P0 (Priority=5) and P2 (Priority=4) suffer from starvation:

- **P0** arrives at $t=0$ but completes at $t=15$ (waited 11ms for 4ms of execution!)
 - P0 was preempted at $t=1$ and had to wait until $t=12$ to resume
 - **Waiting time = 11ms, which is 275% of its burst time!**
- **P2** arrives at $t=2$ but starts execution at $t=11$ (waited 9ms for 1ms of execution!)
 - **Waiting time = 9ms, which is 900% of its burst time!**
- **Low priority processes (P0, P2) starve** while high priority processes (P3, P1, P4) execute
- If more high-priority processes keep arriving, P0 and P2 could wait indefinitely

Preemptive Priority Algorithm - STARVATION PROBLEM with AGING Solution:

Core Idea: Gradually increase priority of waiting processes over time

How Aging Works:

1. Process starts with initial priority
2. Every T time units of waiting → Increase priority
3. Eventually, even lowest priority process becomes highest!
4. Process gets CPU → Reset to original priority

[P0][P1][P1][P3][P3][P3][P3][P3][P1][P2][P4][P4][P0][P0][P0]
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

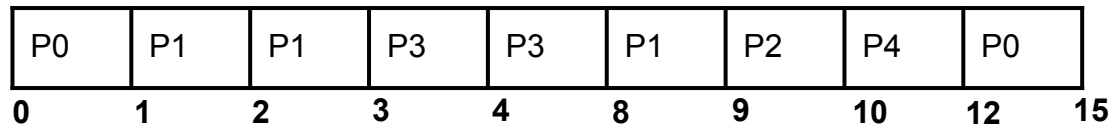
Preemptive Priority Algorithm - STARVATION PROBLEM with AGING Solution:

Pid	AT (ms)	BT (ms)	Priority
P0	0	4	5
P1	1	3	2
P2	2	1	4
P3	3	5	1
P4	4	2	3

Aging Mechanism:

- Every 3ms of waiting time, increase priority by 1 (decrease priority number by 1)
- This prevents starvation by gradually increasing priority of waiting processes

Gantt Chart



Pid	AT	BT	Original Priority	Aged Priority	CT	TAT = CT-AT	WT = TAT - BT
P0	0	4	5	5→4→3→2	15	15	11
P1	1	3	2	2→1	9	8	5
P2	2	1	4	4→3→2	10	8	7
P3	3	5	1	1	8	5	0
P4	4	2	3	3→2→1	12	8	6

MACOS & PRIORITY SCHEDULING:

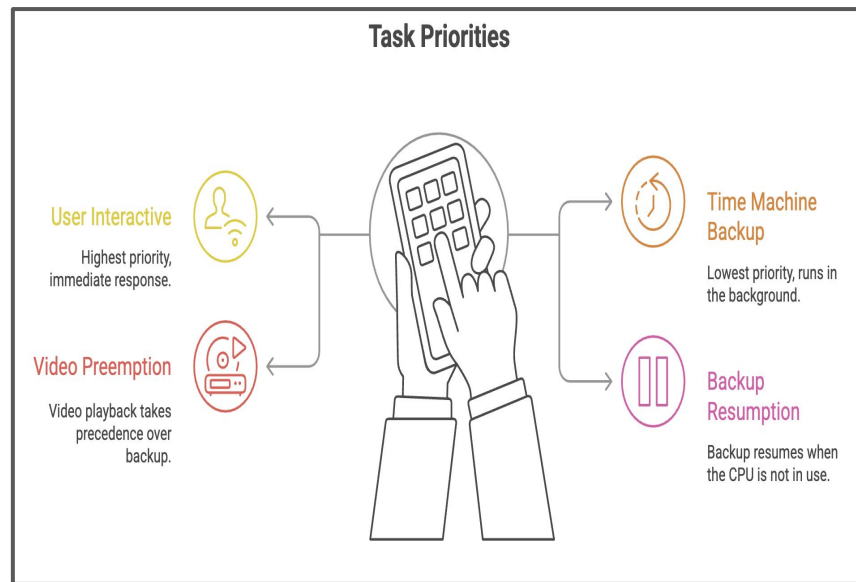
Quality of Service (QoS) Classes

QoS Class	Priority	Use Case	Example
User Interactive	Highest	Direct user actions	Button clicks, animations, typing
User Initiated	High	User-requested tasks	Opening files, search
Default	Medium	General tasks	Network requests, timers
Utility	Low	Background work	Downloads, sync
Background	Lowest	Maintenance tasks	Backups, indexing, cleanup

MACOS & PRIORITY SCHEDULING:

Example: You're watching a video while a backup runs

- Video decoding → **User Interactive** (Priority 1)
- Time Machine backup → **Background** (Priority 5)
- Video **preempts** backup if CPU needed
- Backup continues when CPU idle



What happens:

- Lowers priority of background tasks
- Boosts priority of user-interactive tasks
- Extends battery life while maintaining responsiveness

MULTI-LEVEL QUEUE (MLQ) Algorithm – THE AIRPORT ANALOGY:

Imagine airport security with separate lines:



First Class Line (Queue 1)



Business Class Line (Queue 2)



Economy Line (Queue 3)

Key Rules:

1. Each passenger assigned to ONE line (permanent)
2. First Class always served before Business
3. Business always served before Economy
4. Within same line → FCFS or RR

Can you switch lines? **✗** NO! (In basic MLQ)

This is Multi-Level Queue Scheduling!



First Class Line (Queue 1)

- Shortest wait time
- Dedicated agents
- Best service



Business Class Line (Queue 2)

- Medium wait time
- Shared agents
- Good service



Economy Line (Queue 3)

- Longest wait time
- Regular service
- More passengers

MULTI-LEVEL QUEUE (MLQ) – the “Separate Lines” ALGORITHM

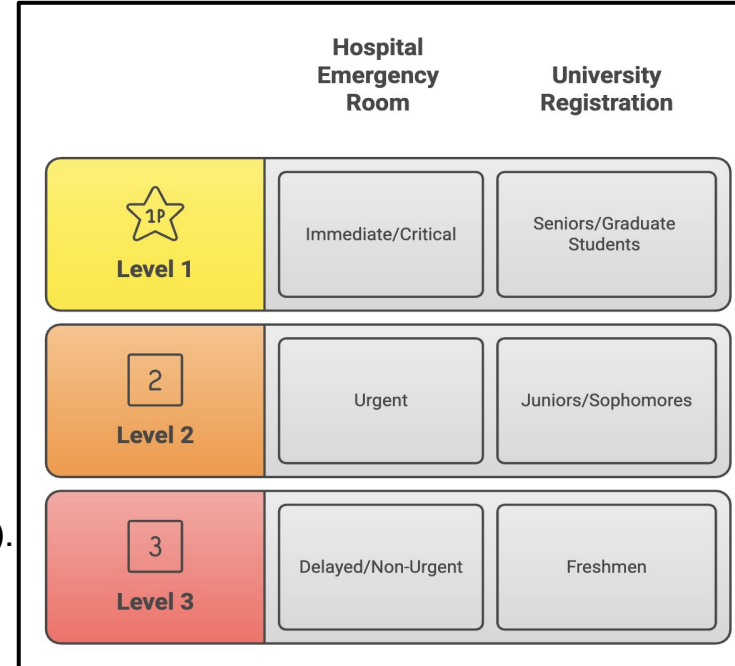
Core Principle:

- Processes divided into separate queues based on type
- Each queue has its own scheduling algorithm
- Fixed priority between queues
- Process stays in assigned queue forever

Mechanism:

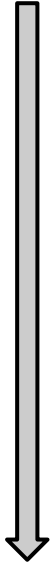
- **Criteria:** Process Type/Class (System, Interactive, Batch).
- **Mode:** Preemptive between queues.
- **Data Structure:** Multiple separate queues.

Example of MLQS



MULTI-LEVEL QUEUE (MLQ) – the “Separate Lines” ALGORITHM

High Response Time
And High Priority



Low Response Time
And Low Priority

**Queue 1: System Processes /
Real-time processes
(FCFS or Priority)**



**Highest Priority
Always runs first**

**Queue 2: Interactive Processes /
Foreground processes
(Round Robin, q = 20ms)**



**High Priority
Runs if Q1 empty**

**Queue 3: Batch Processes /
Background processes
(FCFS)**



**Low Priority
Runs if Q1, Q2 empty**

MULTI-LEVEL QUEUE (MLQ) – the “Separate Lines” ALGORITHM

Starvation ?



MULTI-LEVEL FEEDBACK QUEUE (MLFQ) – THE PROMOTION/DEMOTION SYSTEM

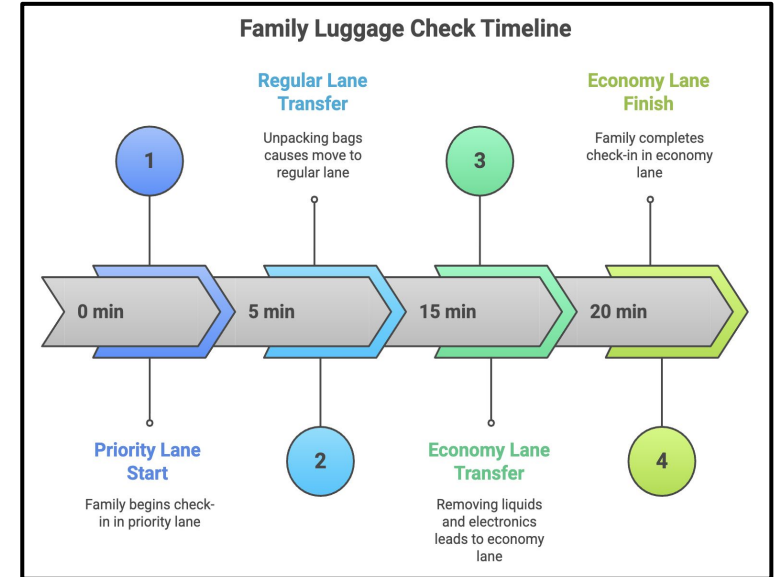
Remember our airport?

What if passengers could change lines?

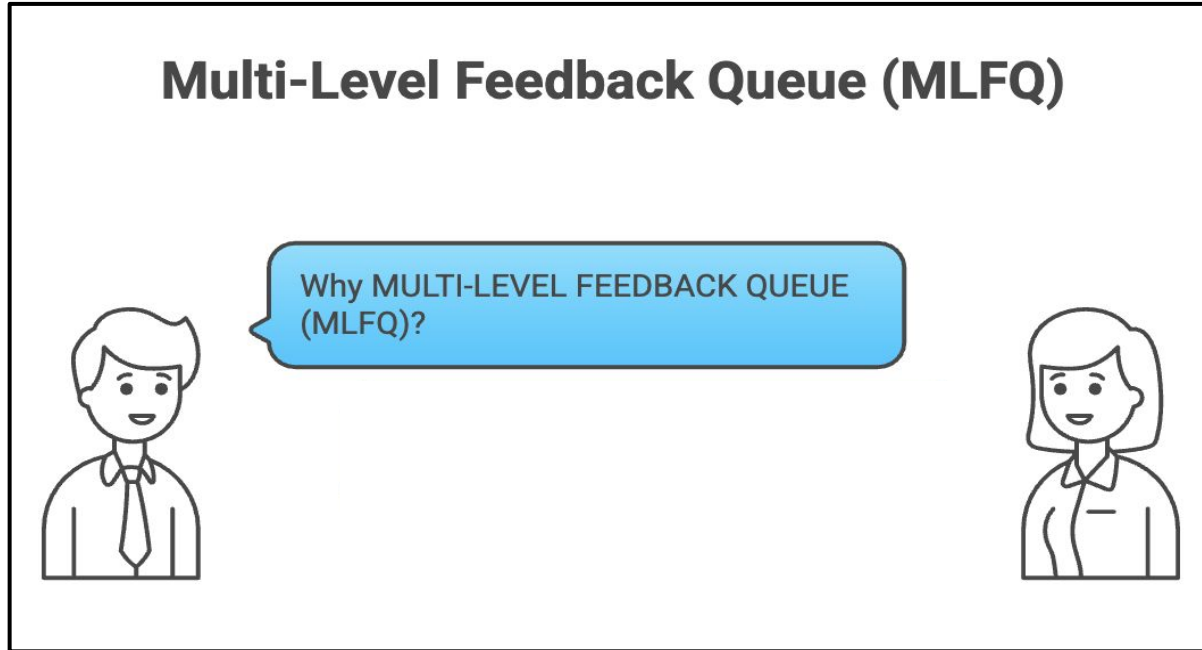
New Rule: Dynamic Line Movement

- **Start in Priority Lane** (fastest service)
- If your security check takes too long → **Demoted to Regular Lane**
- If still too long → **Demoted to Economy Lane**
- Quick passengers stay in Priority Lane!

Example: Family Vacation (Long Check)



MULTI-LEVEL FEEDBACK QUEUE (MLFQ) – THE PROMOTION/DEMOTION SYSTEM



MLFQ - THE "ADAPTIVE LEARNING" ALGORITHM:

Core Principle:

- Multiple queues with different priorities
- Processes can MOVE between queues
- Demotion: Use too much CPU → Move to lower queue
- Promotion: Wait too long OR I/O intensive → Move to higher queue
- System learns process behavior dynamically

Mechanism:

- **Criteria:** CPU usage behavior + Waiting time.
- **Mode:** Preemptive (higher queues preempt lower).
- **Data Structure:** Multiple linked queues with migration.

MLFQ - THE "ADAPTIVE LEARNING" ALGORITHM:

Step 1: Queue Structure Setup - Create multiple queues with different priorities

Queue 0 (Highest Priority): Time Quantum = q

Queue 1 (Medium Priority): Time Quantum = $2q$

Queue 2 (Lowest Priority): Time Quantum = $4q$ (or FCFS)

Step 2: Process Arrival

All new processes enter Queue 0 (highest priority)

Assumption: Every process might be short, give it a chance!

Step 3: Scheduling Rule

Always schedule from highest priority non-empty queue

If Queue 0 has processes $\rightarrow$ schedule from Queue 0

If Queue 0 empty and Queue 1 has processes $\rightarrow$ schedule from Queue 1

If Queue 0 and Queue 1 empty $\rightarrow$ schedule from Queue 2

Step 4: Demotion Rule (Key Feature!)

If process uses its FULL time quantum $\rightarrow$ Demote to next lower queue

If process finishes or blocks (I/O) before quantum expires $\rightarrow$ Stays in same queue (or promoted)

Step 5: Promotion Rule (Optional - Prevents Starvation)

After a certain period T (aging time)

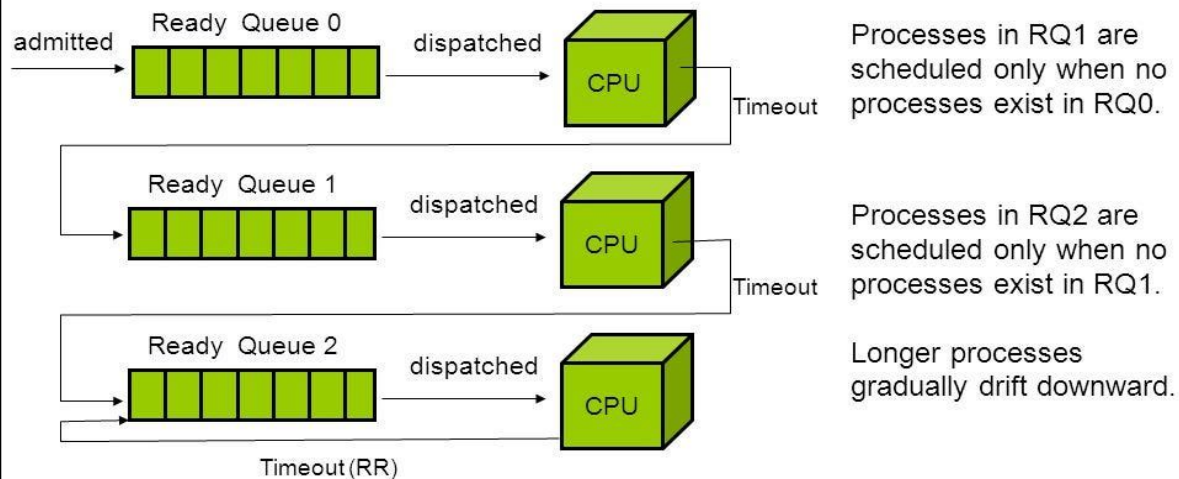
Move all processes back to Queue 0

Prevents low-priority processes from starving

MLFQ - THE "ADAPTIVE LEARNING" ALGORITHM:

Multilevel Feedback Queue Scheduling

- Another way to put a preference on short-lived processes
 - Penalize processes that have been running longer.
- Preemptive



MLFQ - MACOS & MLFQ

macOS Scheduling System:

- Uses MLFQ-based scheduling for process management
- Default: 4 priority queues (Queue 0 to Queue 3)
- Queue 0 = Highest Priority, Queue 3 = Lowest Priority

Queue	Priority	Time Quantum
Q0	Highest	4
Q1	High	8
Q2	Medium	16
Q3	Low	32

MLFQ - MACOS & MLFQ

Priority Hierarchy (Highest to Lowest):

- I. **Real-Time Threads** (Audio, Video) - HIGHEST
 - A. Outside MLFQ, absolute highest priority
- II. **Interactive/Foreground Processes** (Safari, Mail, Finder) - HIGH
 - A. Stay in Q0, Q1 (high-priority queues)
- III. **System Daemons/Services** (Network services, Print services) - MEDIUM
 - A. Usually operate in Q1, Q2 (medium-priority queues)
 - B. Important system tasks that aren't user-facing
- IV. **Background Processes** (Time Machine, Updates, Indexing) - LOW/LOWEST
 - A. Move to Q2, Q3 (low-priority queues)
 - B. Can be delayed without user noticing



Thank You!